ConnectLocal Program Design (Pseudocode)

1. Purpose and Scope

This document describes the planned program logic for ConnectLocal. It is a design-level guide, not production source code. It covers the main customer, provider, and administrator workflows across the Flutter client, Express API, Supabase services, and external integrations.

The backend is a modular monolith: API routes validate and authorize requests, service modules apply business rules, repositories perform database operations, and integrations communicate with Supabase, Google Maps, PayFast, and the push-notification provider.

2. System Entry and Request Handling

```
PROGRAM StartServer
    LOAD environment configuration
    VERIFY required configuration is present
    CONNECT to Supabase and PostgreSQL
    REGISTER request logging, security headers, and body parsing
    REGISTER authentication and validation middleware
    REGISTER API routes for auth, profiles, search, services, bookings,
        payments, messages, reviews, notifications, and administration
    REGISTER centralized error handler
    START HTTP server on configured port
END PROGRAM

HANDLER HandleApiRequest(request)
    route <- FIND route matching request.method and request.path
    IF route does not exist THEN
        RETURN 404 error response
    END IF

    IF route requires authentication THEN
        token <- EXTRACT bearer token from request
        identity <- VERIFY token with Supabase Auth
        IF identity is invalid THEN
            RETURN 401 error response
        END IF
        request.user <- LOAD user identity and role
    END IF

    IF user's role is not permitted by route THEN
        RETURN 403 error response
    END IF

    IF request body or parameters fail route validation THEN
        RETURN 400 validation error
    END IF

    result <- CALL route controller with validated request
    RETURN formatted success response containing result
END HANDLER
```

3. Authentication and Access Control

```
PROCEDURE RegisterWithEmail(email, password, profileData, requestedRole)
    VALIDATE email and password
    IF requestedRole is not an allowed public role THEN
        RETURN validation error
    END IF

    authUser <- CREATE Supabase Auth account
    IF account creation fails THEN
        RETURN registration error
    END IF

    CREATE application profile for authUser
    SET profile role to requestedRole
    SEND verification email
    RETURN account created with verification required
END PROCEDURE

PROCEDURE Login(credentials)
    session <- AUTHENTICATE through Supabase Auth
    IF credentials are invalid THEN
        RETURN 401 authentication error
    END IF

    profile <- LOAD application profile for session user
    IF email verification is required and not complete THEN
        RETURN verification-required response
    END IF

    RETURN access token, refresh token, and permitted profile data
END PROCEDURE

PROCEDURE AuthorizeRequest(request, permittedRoles)
    identity <- VERIFY Supabase access token
    IF identity is invalid or expired THEN
        DENY request with 401
    END IF
    profile <- LOAD identity's application profile
    IF profile.role is not in permittedRoles THEN
        DENY request with 403
    END IF
    RETURN identity and profile
END PROCEDURE
```

4. Provider Discovery

```
PROCEDURE SearchProviders(customer, location, radius, filters, page)
    VALIDATE location, radius, filters, and page
    IF location is an address rather than coordinates THEN
        location <- GEOCODE address with Google Maps
    END IF

    providers <- QUERY database for providers that are:
        publicly listed, verified, within radius, and offering active services
    providers <- APPLY category, price, and rating filters
    providers <- SORT by distance and selected sort option
    pageResults <- PAGINATE providers using page settings

    FOR EACH provider IN pageResults
        provider.distance <- CALCULATE or LOOK UP distance
        provider.travelEstimate <- REQUEST travel estimate when available
    END FOR

    RETURN pageResults and pagination metadata
END PROCEDURE
```

5. Booking Lifecycle

```
PROCEDURE RequestBooking(customer, providerId, serviceId, scheduledStart, notes)
    REQUIRE customer is authenticated with role customer
    VALIDATE service is active and belongs to providerId
    VALIDATE scheduledStart is in the future and within provider availability
    price <- READ current service price from database

    BEGIN DATABASE TRANSACTION
        LOCK provider schedule for the requested interval
        IF an active booking conflicts with the requested interval THEN
            ROLLBACK transaction
            RETURN 409 slot-unavailable error
        END IF

        booking <- INSERT booking with:
            customer, provider, service, scheduledStart, price, notes,
            status pending, and payment status pending
    COMMIT transaction

    SEND provider a new-booking notification
    RETURN booking
END PROCEDURE

PROCEDURE RespondToBooking(provider, bookingId, action, reason)
    booking <- LOAD booking and verify provider is its assigned provider
    REQUIRE booking.status is pending

    IF action is accept THEN
        CONFIRM slot is still available using an atomic database operation
        UPDATE booking status to confirmed
    ELSE IF action is decline THEN
        UPDATE booking status to declined and record reason
    ELSE
        RETURN 400 invalid-action error
    END IF

    SAVE status change
    NOTIFY customer of the result
    RETURN updated booking
END PROCEDURE

PROCEDURE RescheduleBooking(customer, bookingId, newScheduledStart)
    booking <- LOAD booking and verify customer is a booking party
    REQUIRE booking may be rescheduled under current policy
    VALIDATE newScheduledStart against provider availability

    BEGIN DATABASE TRANSACTION
        LOCK provider schedule for the new interval
        IF new interval conflicts with another active booking THEN
            ROLLBACK transaction
            RETURN 409 slot-unavailable error
        END IF
        UPDATE booking scheduledStart to newScheduledStart
        SET booking status to pending for provider re-confirmation
    COMMIT transaction

    NOTIFY provider that confirmation is required
    RETURN updated booking
END PROCEDURE

PROCEDURE CancelBooking(actor, bookingId, reason)
    booking <- LOAD booking and verify actor is a booking party
    REQUIRE booking is not completed, cancelled, or declined
    ENFORCE configured cancellation policy
    UPDATE booking status to cancelled and record reason
    RELEASE the provider's reserved time slot
    NOTIFY the other booking party
    RETURN updated booking
END PROCEDURE

PROCEDURE CompleteBooking(provider, bookingId)
    booking <- LOAD booking and verify provider owns the booking
    REQUIRE booking status is confirmed or in_progress
    UPDATE booking status to completed
    NOTIFY customer that a review can be submitted
    RETURN updated booking
END PROCEDURE
```

6. Payment and PayFast Confirmation

```
PROCEDURE InitiatePayment(customer, bookingId)
    booking <- LOAD booking and verify customer owns it
    REQUIRE booking status is confirmed
    REQUIRE no completed payment already exists for booking

    payment <- CREATE payment record with pending status and booking amount
    redirectUrl <- CREATE PayFast hosted-payment request for payment
    RETURN payment identifier, amount, and redirectUrl
END PROCEDURE

HANDLER ProcessPayFastITN(notification)
    VALIDATE required callback fields are present
    IF payload is malformed THEN
        RETURN 400
    END IF

    IF callback signature is invalid OR source IP is not allowed THEN
        RECORD rejected callback without exposing validation details
        RETURN plain-text OK as required by the callback contract
    END IF

    BEGIN DATABASE TRANSACTION
        payment <- LOAD payment identified by callback with a row lock
        IF payment does not exist THEN
            RECORD callback for investigation
            COMMIT transaction
            RETURN plain-text OK
        END IF

        IF payment is already final THEN
            COMMIT transaction
            RETURN plain-text OK
        END IF

        IF callback amount or currency does not match expected payment THEN
            RECORD payment discrepancy
            ROLLBACK transaction
            RETURN plain-text OK
        END IF

        IF callback status is COMPLETE THEN
            SET payment status to paid
            SET linked booking payment status to paid
        ELSE IF callback status indicates failure THEN
            SET payment status to failed
        END IF
    COMMIT transaction

    NOTIFY customer and provider of the payment result
    RETURN plain-text OK
END HANDLER
```

7. Messaging and Reviews

```
PROCEDURE SendMessage(sender, conversationId, messageText)
    REQUIRE sender is a participant in conversationId
    VALIDATE messageText is non-empty and within size limits
    message <- SAVE message to database
    PUBLISH message to authorized Supabase Realtime channel
    CREATE notification for recipient, respecting notification settings
    RETURN saved message
END PROCEDURE

PROCEDURE SubmitReview(customer, bookingId, rating, reviewText)
    booking <- LOAD booking and verify customer owns it
    REQUIRE booking status is completed
    REQUIRE no review already exists for booking
    VALIDATE rating is an integer from 1 through 5
    SAVE review with moderation status visible
    RECALCULATE provider's displayed average rating
    RETURN saved review
END PROCEDURE

PROCEDURE ModerateReview(admin, reviewId, decision)
    REQUIRE admin role
    REQUIRE decision is hide or restore
    UPDATE review visibility and record moderation audit details
    RETURN updated review
END PROCEDURE
```

8. Provider Verification and Administration

```
PROCEDURE SubmitProviderVerification(provider, documents)
    REQUIRE provider role
    VALIDATE required identity and business documents
    STORE documents in protected storage
    CREATE verification submission with status pending
    NOTIFY administrators of pending submission
    RETURN submission status
END PROCEDURE

PROCEDURE ReviewProviderVerification(admin, submissionId, decision, notes)
    REQUIRE admin role
    submission <- LOAD pending verification submission
    REQUIRE decision is approve or reject
    UPDATE submission status and record reviewer and notes
    IF decision is approve THEN
        MARK provider verified and eligible for public listing
    ELSE
        KEEP provider hidden and provide rejection outcome
    END IF
    NOTIFY provider of the decision
    RETURN updated submission
END PROCEDURE

PROCEDURE SuspendAccount(admin, userId, reason)
    REQUIRE admin role
    user <- LOAD target user
    REQUIRE reason is provided
    SET account status to suspended
    REVOKE or invalidate active sessions where supported
    RECORD administrative audit event
    NOTIFY affected user
    RETURN updated account status
END PROCEDURE
```

9. Notifications and Error Handling

```
PROCEDURE NotifyUser(userId, event, details)
    preferences <- LOAD user's preferences for event category
    SAVE in-app notification unless disabled by policy
    IF push notifications are enabled for this category THEN
        SEND push notification through configured provider
    END IF
    RECORD delivery outcome for operational review
END PROCEDURE

HANDLER HandleApplicationError(error, request)
    WRITE structured error details to server log
    IF error is a known validation, authorization, conflict, or not-found error THEN
        RETURN its appropriate HTTP status and safe public message
    ELSE
        RETURN 500 with a generic message and request correlation identifier
    END IF
    NEVER return secrets, tokens, or internal stack traces to the client
END HANDLER
```

10. Design Rules

- The API is authoritative for authorization, validation, prices, booking states, and payment states; client-side checks only improve usability.
- Database transactions and appropriate row or schedule locks protect booking slots and payment updates from concurrent requests.
- State-changing operations verify both the caller's role and ownership of the affected record.
- PayFast callback processing is server-to-server, signature-checked, IP-checked, amount-checked, and safe to receive more than once.
- Secrets remain in environment configuration and are never returned to clients or written to ordinary logs.
- External-service failures must not falsely confirm a booking or payment; record the failure and return a controlled response.